
Efficient Ensemble Inference

Nick Kocher¹ Janek Paeßens¹ Anja Jankovic¹ Holger H. Hoos^{1,2,3}

¹RWTH Aachen University

²Leiden University

³University of British Columbia

Abstract Ensembling improves predictive performance in AutoML systems but often leads to inefficient inference on CPUs, as most frameworks execute ensemble members sequentially while allocating all available CPU cores to each model. Since inference scales poorly beyond a few cores and ensemble members vary in running time, this wastes both time and energy. We address this by framing ensemble inference as a moldable task scheduling problem, where CPU core allocation and execution order for inference time and energy consumption are jointly optimised. Building on a two-stage allotment and scheduling framework, we introduce NSGA-P, a multi-objective allotment strategy based on NSGA-II as an analytical tool to understand the benefits of efficient ensemble inference using offline resource measurements. We evaluate our approach on 104 datasets from the AMLB benchmark suite. NSGA-P outperforms the naïve maximum-cores baseline used by existing AutoML frameworks in over 96% of ensembles. NSGA-P allotments halve inference time on 8 cores and achieve up to a 74% reduction in both inference time and energy consumption on 64 cores. These results demonstrate that intelligent parallel resource allocation can substantially improve the efficiency of resource-aware AutoML systems.

1 Introduction

Ensembling is fundamental to modern Automated Machine Learning (AutoML) systems, consistently delivering strong predictive performance by combining multiple base models (Erickson, Mueller, et al., 2020; Feurer et al., 2022; Hernandez et al., 2025; LeDell and Poirier, 2020; Thornton et al., 2013). As these systems mature, the computational cost of inference has become a crucial concern, particularly in deployment settings, such as in energy-constrained or edge devices, where latency budgets are limited and energy efficiency is as important as predictive accuracy. The computing hardware used in such settings often lacks access to expensive dedicated Graphical Processing Units (GPUs) and thus have to rely on Central Processing Units (CPUs) for inference.

A critical but underexplored aspect of ensemble inference in this setting is how computational resources are assigned to individual ensemble members. Most AutoML frameworks, including Auto-sklearn 2.0 (Feurer et al., 2022) and AutoGluon (Erickson, Mueller, et al., 2020), implicitly adopt a naïve strategy, where each model receives the full available core count and ensemble members are run sequentially. This approach ignores two fundamental facts. Firstly, parallel scaling of machine learning inference is sublinear: doubling the core count does not halve inference time, and beyond a few cores, additional cores yield diminishing speed-up, while energy draw grows approximately linearly. As a concrete example, on the AMD EPYC 7543 platform used in this work, going from one to two cores yields the largest marginal speed-up, whereas going from four to eight cores provides far less benefit at twice the energy cost (see Figure 1). Secondly, when ensemble members differ in size and complexity, homogeneous core allocation, such as allocating one core to each model, causes many cores to sit idle waiting for a single long model evaluation to finish, leading to unnecessarily increased total running time.

Here, we address these issues by framing ensemble inference as a moldable task scheduling problem: Each ensemble member is a task whose core allocation can be chosen prior to execution,

and the goal is to jointly optimise the allotment and execution order to minimise inference time and energy consumption. To this end, we exploit a key structural property of ensemble inference. Because all ensemble members are fixed after training, their inference times and energy consumption at each core count can be measured once on the training data used for the ensembling stage, incurring only a one-time overhead that is negligible relative to training time, as state-of-the-art tabular models typically have inference times one to two orders of magnitude lower than their training times (Erickson, L. Purucker, et al., 2025). For ensembles used repeatedly in production, the initial profiling cost is easily amortised over many inference calls, so it often becomes negligible in practice. These offline measurements then serve as a lookup table, meaning that all subsequent allotment and scheduling decisions are made entirely in simulation, without requiring any further runs of the model.

Building on the two-stage allotment and scheduling framework of Benoit et al. (2022), we replace the allotment stage with NSGA-P, an NSGA-II-based multi-objective allotment strategy that searches over per-member core counts to approximate efficient execution schedules for ensembles. The main contribution of this work is not the allotment algorithm itself, but the simulation-based framework as an analytical tool and the resulting empirical characterisation of the conditions under which heterogeneous core allocation yields practical gains.

We empirically evaluate our approach on the AMLB benchmark suite, comprising 71 classification and 33 regression tasks. First, we show that parallelisation exhibits a clear and consistent trade-off: inference time decreases sharply from 1 to 2 cores, then stagnates, while energy cost grows roughly linearly with core count, implying that naïve maximum-core allocation is rarely energy-efficient. Second, we demonstrate that resource measurements taken during ensembling transfer reliably to final prediction ($R^2 > 0.91$), validating offline simulation as a sound basis for scheduling decisions. Third, our proposed NSGA-P allotment strategy dominates the naïve maximum-cores baseline for 96% of the ensembles we considered, achieving up to a 56% reduction in inference time and 27% reduction in energy consumption on 8 cores, with gains increasing further for higher core counts, with up to 74% reduction in both inference time and energy consumption on 64 cores. These results establish that core allocation for ensemble inference together with principled scheduling yields substantial, practical gains over the current state of practice. Code and data for our experiments are available at: https://github.com/NickKocher/efficient_ensemble_inference.

The remainder of the paper is structured as follows: In Section 2, we introduce relevant background concepts on ensembling and multi-objective optimisation, and highlight related work relevant to the allotment and scheduling process. In Section 3, we present our framework for efficient scheduling of ensembles. In Section 4, we introduce the setup of our experiments, before we present and analyse the results we obtained in Section 5. Finally, Section 6 provides concluding remarks and discusses future work.

2 Preliminaries and Related Work

In this section, we position our study with respect to prior work in ensembling, allotment and scheduling, and provide relevant background for multi-objective optimisation.

2.1 Ensembling in AutoML

Multiple studies have shown that combining models through ensembling leads to improved predictive performance and may even be more efficient than investing more resources into longer training of individual models (Erickson, Mueller, et al., 2020; Feurer et al., 2022; Hernandez et al., 2025; LeDell and Poirier, 2020; Thornton et al., 2013). Post-hoc ensembling methods, which combine models after independent training, are standard in AutoML pipelines due to the modularity arising from independently trained models. One of the most widely used post-hoc ensembling methods is

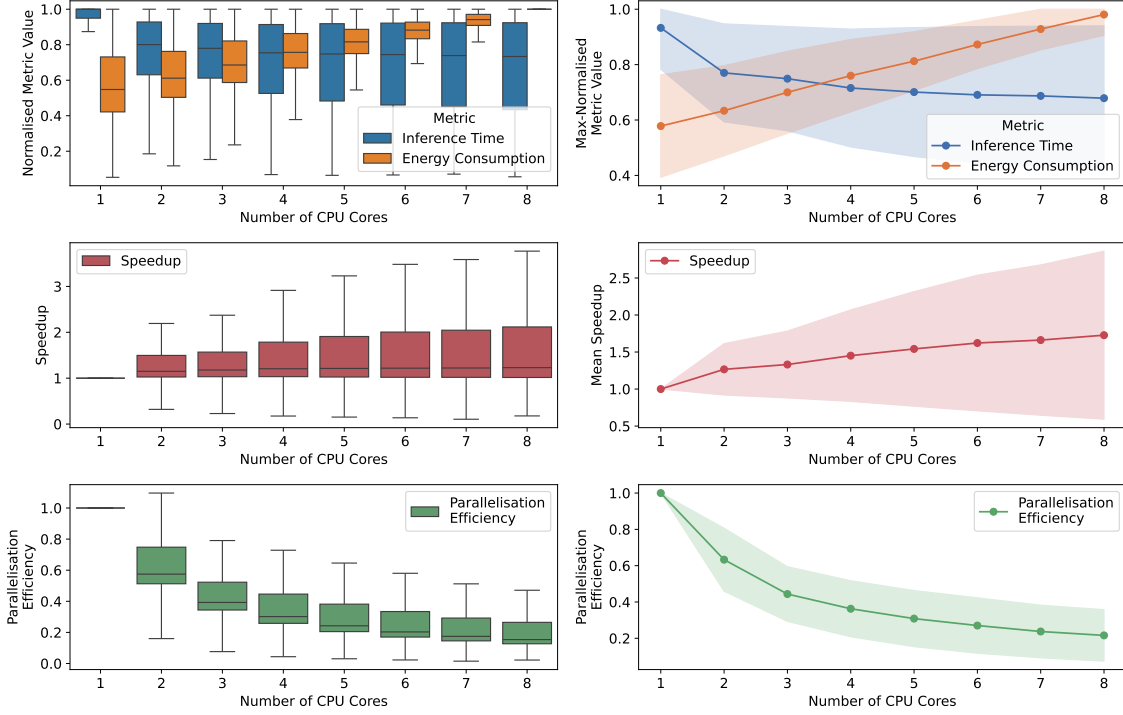


Figure 1: Trade-offs between energy consumption and inference time due to parallelisation on the AMD EPYC 7543 platform. Results are aggregated across all datasets and models. Models were randomly sampled from the search space. Inference times and energy consumptions are normalised to the maximum observed value per model and dataset. Top: Normalised inference time vs normalised energy consumption; centre: Speed-up; bottom: Energy efficiency.

greedy ensemble selection (GES) (Caruana et al., 2004), which greedily selects models that maximally improve ensemble performance and produces sparse ensembles, thus reducing inference cost. Before ensembling, the model pool is pruned to a fixed number of models, commonly to the Top- N selection per-family (Silo-Top- N ; L. O. Purucker et al., 2023), to keep the ensembling process tractable.

More recent work has introduced population-based post-hoc ensemble selection. Quality Optimisation Ensemble Selection (QO-ES) and Quality Diversity Optimisation Ensemble Selection (QDO-ES) (L. O. Purucker et al., 2023) maintain populations of ensembles, optimised using evolutionary approaches, with QDO-ES additionally promoting diversity characterised by average loss correlation and configuration space similarity. Maier et al. (2024) extended this with resource-aware variants (Infer-QDO-ES, Size-QDO-ES) that substitute configuration space similarity with inference time or ensemble size; this is the prior work most closely related to ours. We complement this line of research by optimising *how* a fixed ensemble is executed, rather than which models constitute it.

2.2 Allotment and scheduling

Multithreaded ensemble inference raises two nested problems: (1) rigid scheduling: ordering ensemble members with fixed core counts to minimise makespan, *i.e.*, the total running time required to run all models; and (2) joint allotment and scheduling: choosing per-member core counts and their execution order. Both are instances of multiprocessor task scheduling (Drozdowski, 1996), a problem known to be $\mathcal{NP}$ -complete even in the rigid case (Du and Leung, 1989). We focus on the *offline* setting, where task durations are known in advance from initial profiling, as opposed

to online scheduling, where tasks arrive on-the-fly (Villarrubia et al., 2025). A related but distinct line of work addresses offline algorithm scheduling under a fixed single-core time budget, as in ASPEED (Hoos et al., 2012). However, this setting does not account for per-task core allocation, which is the primary aspect optimised in our framework, and extending ASPEED-like approaches to the moldable, multi-objective setting would require a reformulation of the underlying optimisation problem, which is outside the scope of this study.

A scheduling algorithm is said to be a ρ -approximation if it produces a makespan at most ρ times the optimal makespan for any problem instance; smaller ρ indicates a stronger worst-case guarantee. For rigid tasks, list scheduling algorithms (Garey and Graham, 1975) greedily assign tasks ordered by a priority criterion (*e.g.*, longest processing time, highest core count, or highest workload; Dutot et al., 2004), achieving an approximation ratio of 2 regardless of the criterion used (Turek et al., 1992). For moldable tasks, where core counts are chosen prior to execution (Dutot et al., 2004), two prominent approaches exist. LPA-List (Benoit et al., 2022) uses a two-stage procedure: a local processor allocation (LPA) strategy assigns core counts by minimising a combined workload-and-time penalty, followed by list scheduling of the resulting rigid tasks. It achieves approximation ratios of 2 (linear speed-up), 3 (communication-time model), and $\mathcal{O}(\sqrt{N})$ (monotonic speed-up), and can accommodate additional objectives. A Polynomial-Time Approximation Scheme (PTAS) by Jansen (2012) achieves a tighter $1.5 + \epsilon$ ratio under monotonic speed-up, but is restricted to the single-objective case.

2.3 Multi-objective optimisation

Many real-world optimisation problems involve multiple competing objectives that cannot be simultaneously minimised by a single solution. In such settings, the goal is to find the Pareto set, *i.e.*, the set of solutions for which no objective can be improved without degrading at least one of the others. A solution x is said to dominate another solution y if and only if, it is at least as good in all objectives and strictly better in at least one. The Pareto set consists of all non-dominated solutions, and its mapping to the objective space corresponds to the Pareto front.

Evolutionary algorithms are well-suited to multi-objective optimisation, because their population-based mechanism allows an entire Pareto front approximation to be maintained across generations. The Non-dominated Sorting Genetic Algorithm II (NSGA-II) algorithm by Deb et al. (2002) is among the most widely used such algorithms and has strong theoretical and empirical support. NSGA-II maintains a population of candidate solutions, which it evolves over a fixed number of generations using three key mechanisms. Non-dominated sorting partitions the population into ranked fronts, where the first front contains all non-dominated solutions, the second contains solutions dominated only by those in the first front, and so on. Crowding distance then assigns each candidate solution a score that reflects how isolated the solution is in objective space relative to its neighbours on the same front, thereby promoting diversity along the Pareto front by preferring solutions in less crowded regions. Finally, elitist selection ensures that the best solutions found so far are carried forward, by combining parent and offspring populations before selection, so that high-quality solutions are never discarded. Offspring are generated through crossover and mutation, adaptable to the problem at hand. In this work, we consider two objectives: inference time and energy consumption; these are competing, since reducing inference time by allocating more cores to a model typically increases energy consumption.

3 Methodology

To study the potential benefits of heterogeneous execution of ensemble members, we adopted the two-stage allotment and scheduling approach by Benoit et al. (2022) and replaced its allotment stage with an NSGA-II-based strategy to approximate efficient execution schedules for ensembles. Our goal is not to introduce a production-ready scheduler, but rather to provide a simulation-based

analytical framework for understanding how different core-allocation strategies affect ensemble inference efficiency. We left the list scheduling stage unmodified, *i.e.*, minimising makespan automatically minimises idle time and energy consumption for a fixed allotment.

Our method assumes that ensembles have already been constructed through a standard post-hoc procedure, outlined in Section 4. Before any allotment or scheduling decisions are made, we exhaustively profile all ensemble members offline: for each ensemble member, each dataset, and each admissible core count, we measure inference time and energy consumption on both training and test data. This resource measurement pass is the only stage that requires actual parallel execution. All subsequent allotment and scheduling steps operate entirely in simulation, using the offline measurements as a lookup table. Given a candidate allotment, *i.e.*, a mapping from ensemble members to core counts, the list scheduling algorithm uses training-data measurements to simulate the execution order and compute estimated makespan and energy consumption, without running any models. Final inference time and energy consumption are reported by replaying the same schedule using test-data measurements. This simulation on test-data measurements provides an upper bound on achievable efficiency, since process-creation overhead is not accounted for.

We consider three allotment strategies: (1) **Minimum Cores**, where each model is allotted a single core, serving as a baseline to quantify the impact of scheduling; (2) **Maximum Cores**, where each model is allotted the highest available number of cores, serving as a baseline for parallelisation, which corresponds to the current state of the art in many AutoML frameworks; and (3) **NSGA-P**, our novel NSGA-II-based allotment strategy that optimises the allocation of computational resources in terms of the degree of parallelisation for each ensemble member.

In NSGA-P, each candidate solution is encoded as a vector of dimensionality n , where n is the number of ensemble members and represents the core counts allocated to each ensemble member. The two objectives to minimise, total (simulated) inference time and total (simulated) energy consumption of the ensemble, are evaluated in each NSGA-II iteration by running the list scheduling simulation on training-data measurements. Predictive performance is unaffected by the allotment and need not be recomputed during the search.

For all three allotment strategies, the resulting (rigid) tasks are scheduled using list scheduling with the Longest Processing Time First (LPT) priority criterion, using training-data inference times to determine task ordering.

4 Setup of Experiments

This section describes the experiments conducted to empirically evaluate the joint allotment and scheduling of ensembles, outlining the design choices and implementation details.

Datasets. The experiments were conducted on the datasets from the Automated Machine Learning Benchmark (AMLB) benchmark suite (Gijssbers et al., 2024), which includes a diverse set of 71 classification and 33 regression tasks. Data was partitioned using an 80/20 train-test split, with the test set held out exclusively for final evaluation. The training portion was further divided for ensemble member generation and for ensembling: 75% was used for 5-fold cross-validation during ensemble member generation, while the remaining 25% served as hold-out data for ensembling. To maximise the data available at the ensembling stage, we then incorporated the full training set, including the previously held-out portion, achieving a trade-off between the risk of overfitting and having sufficient data for ensembling. For classification tasks, all splits were stratified to preserve class balance, with each split guaranteed to contain at least one instance of every class.

Search space of ensemble members. The search space comprises three model families with complementary inductive biases and resource characteristics: XGBoost (Chen and Guestrin, 2016), RealMLP (Holzmüller et al., 2024) and TabM (Gorishniy et al., 2025); all models were interfaced through the pytabkit framework (Holzmüller et al., 2024). The models were chosen as state-of-the-

art representatives of CPU-executable models on tabular data. Additional preprocessing details can be found in Appendix B. Hyperparameter spaces are listed in Appendix C.

Ensemble member generation. To generate ensemble members, we used the state-of-the-art general-purpose Sequential Model-based Algorithm Configuration (SMAC) framework (Lindauer et al., 2022), with a random forest surrogate and expected improvement as the acquisition function. Each SMAC run used a wall-clock budget of 8 hours, with individual cross-validation runs capped at 5% of the total budget (24 minutes). SMAC was executed with five random seeds per dataset to account for stochasticity. Predictive performance was measured using Receiver Operating Characteristic - Area Under the Curve (ROC-AUC) for binary classification, log-loss for multi-class classification, and Root Mean Squared Error (RMSE) for regression, following Gijssbers et al. (2024).

Pruning and post-hoc ensemble selection. The resulting pool of models was pruned using Silo-Top-50, which selects the top 50 models with regards to predictive performance evenly across model families. Ensembles were then constructed using the de-facto standard post-hoc ensembling approach GES (Caruana et al., 2004) with a maximum ensemble size of 50.

Offline resource measurements. Both objectives, inference time and energy consumption, were normalised on a per-dataset basis. Prior to any allotment and scheduling, inference time and energy consumption were measured for each ensemble member under all admissible core counts, namely from 1 to 8 cores in increments of 1, and from 12 to 64 cores in increments of 4, on both training and test data. Using these admissible per-model core allocations, we evaluated ensemble scheduling under total available core budgets of 2, 4, 8, 16, 32, and 64 cores. Measurements used model parallelisation native to pytabkit, with thread counts capped at the allocated core count and hyperthreading disabled for consistent availability of cores.

Allotment and scheduling. All allotment and scheduling decisions were made in simulation, using the offline measurements described in Section 3. For the NSGA-P allotment, we ran NSGA-II with a population size of 25 and a budget of 50 iterations; these values provide a practical trade-off between computational overhead and adequate exploration of the Pareto front. Crossover was uniform, independently inheriting from one of the two parent solutions with equal probability. Mutation was performed using a tailored operator: core allocations were incremented or decremented by one step (of size 1 for 1 to 8 cores, and of size 4 for 12 to 64 cores), constrained by predefined minimum and maximum core counts. Parent selection used binary tournament. We scheduled all allotments using list scheduling, with the priority criterion set to LPT using training-data inference times to determine execution order (Appendix A.1). Resource consumption for the final comparison was obtained by replaying the simulated schedule on test-data measurements, without any live parallel execution at this stage. For idle cores during the execution, we added the energy consumed by an idle core to the total energy consumption.

Hardware and software details. All experiments were carried out on a server equipped with two AMD EPYC 7543 32-core processors (AMD, 2024) and 1 TB of total RAM, running Rocky Linux 9.3 (Kernel 5.14.0). Unless stated otherwise, ensemble member generation used a single CPU core and 16 GB RAM, with additional resources allocated for large datasets according to a size-based heuristic detailed in Appendix B. Energy measurements were taken using the `amd_energy` driver,¹ which enables per-core energy measurements. While this allows for parallel experiments with isolated measurements per core, the values exclude system-wide components, such as memory and I/O, and therefore represent a lower bound on total energy usage (Kocher et al., 2025; Schöne et al., 2021).

¹https://github.com/amd/amd_energy

5 Results

In this section, we present the results of the empirical evaluation of our ensemble scheduling approach. We first analyse the trade-offs introduced by parallelisation in terms of inference time and energy consumption. We then evaluate the reliability of offline resource profiling for scheduling by comparing measurements obtained during ensembling (on training data) and final prediction (on test data). Finally, we compare the proposed NSGA-P allotment strategy against the minimum-core and maximum-core baselines.

5.1 Potentials and Limits of Parallelisation

To study the effects of parallelisation on the inference time and energy consumption of ensemble members in an unbiased manner, we evaluated randomly sampled models from the search space across all datasets under varying core counts. We used the same setup as for the ensemble member generation using SMAC. Figure 1 summarises the resulting trade-offs between inference time and energy consumption for randomly sampled models across all datasets under various levels of parallelisation, for executions ranging from one to eight CPU cores.

As expected, we observe that increasing the level of parallelisation consistently reduces inference time, albeit with strongly diminishing returns as more cores are added. The largest inference time reduction occurs when moving from one to two cores, whereas additional cores provide progressively smaller speed-ups. However, the reduction in inference time comes at the cost of increased energy consumption, as more cores lead to higher overall power usage during execution. Notably, the increase in energy consumption is approximately linear with respect to the number of cores. Consequently, parallelisation introduces a clear latency-energy trade-off, where initial increases in parallelisation yield substantial inference time improvement for relatively modest energy costs, while aggressive parallelisation becomes increasingly energy-inefficient.

These observations have important implications for ensemble scheduling. When ensembles contain sufficiently many models that are homogeneous in terms of their inference time, allocating a single core per ensemble member is often the most energy-efficient strategy, as parallelism is exploited at the ensemble level. However, for smaller ensembles or ensembles with highly heterogeneous inference times, assigning additional cores to certain ensemble members, *i.e.*, a higher level of parallelisation on the member level, can reduce scheduling gaps and improve both inference time and overall efficiency.

5.2 Consistency between Resources during Ensembling and during Final Prediction

A key assumption of our simulation-based scheduling approach is that resource measurements collected during ensembling transfer reliably to final prediction, *i.e.*, there is consistency between the resources used for the offline creation of the allotments and schedules on training data and the resources used for predictions on test data. If this assumption does not hold, allotment and scheduling decisions derived from offline measurements may not accurately reflect actual resource consumption in deployment. This can lead to suboptimal heuristics for scheduling, since the LPT heuristic uses training inference time as an estimate for inference time during scheduling.

To test this assumption, we measured inference time and energy consumption for randomly sampled models across all datasets for one to eight cores during both ensembling and final prediction phases on the AMD EPYC 7543 platform. Figure 2 compares the corresponding measurements for single-core execution. Overall, we observe a high degree of consistency between the two phases. Inference time measurements achieve an R^2 score of 0.941 and a Spearman correlation of 0.978, while energy consumption measurements achieve an R^2 score of 0.941 and a Spearman correlation of 0.989. Similar trends were observed across other core counts (Appendix A.3). These results indicate that the resource measurements obtained during ensembling generalise reliably to those obtained during final predictions, validating offline simulation as a practical basis for allotment and scheduling decisions.

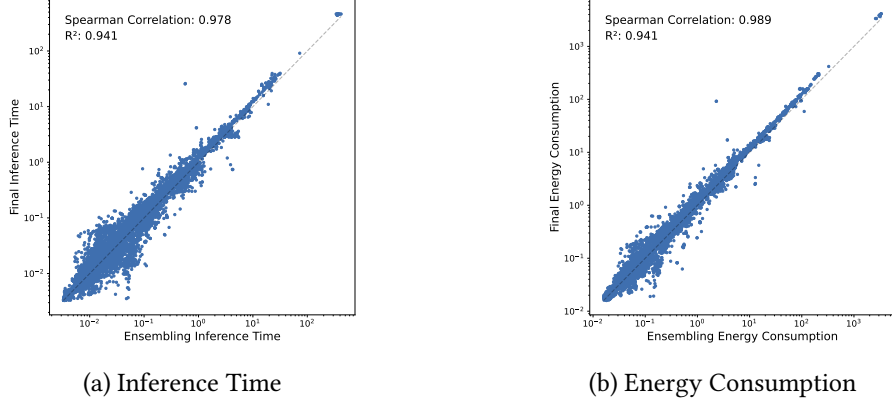


Figure 2: Consistency of inference time and energy consumption between the ensembling stage (on training data) and final predictions (on test data) using one core of the AMD EPYC 7543 platform. Values are normalised per model and dataset.

5.3 Allotment and Scheduling

We now compare the three allotment strategies considered in this work: Maximum Cores, corresponding to the standard strategy used in many existing AutoML frameworks (Erickson, Mueller, et al., 2020; Feurer et al., 2022); Minimum Cores, where each model receives a single core; and the proposed NSGA-P multi-objective allotment strategy. As NSGA-II produces multiple trade-offs between inference time and energy consumption, we report the allotment yielding the greatest reduction in inference time along with its associated energy reduction.

We present the main results in Figure 3. NSGA-P identifies at least one allotment that dominates the Maximum Cores baseline with respect to both computational resources in 96.72% of ensembles across all core counts, increasing to 99.9% for 64 available cores. The average inference time reduction achieved by NSGA-P increases monotonically from 23.5% at 2 cores to 73.73% at 64 cores, while the average reduction in energy consumption increases from 5.24% to 74.45% between 2 and 64 cores. In comparison, the Minimum Cores strategy achieves significantly smaller inference time reductions (from 21.84% to 46.53%), particularly at higher core counts, despite achieving comparable reductions in energy consumption (from 5.28% to 73.82%).

A key advantage of NSGA-P is its stability across heterogeneous ensembles. While the Minimum Cores strategy can perform decently on average, it exhibits substantially higher variance in inference time, due to scheduling bottlenecks caused by long-running models. For instance, at 64 available cores, the standard deviation of inference time reduction is 95.43% for Minimum Cores, compared to 22.16% for NSGA-P. In contrast, when it comes to energy consumption, the variance remains similar between the two strategies, and there is little or no observed difference between reductions in energy achieved by these two strategies.

Using the Wilcoxon signed-rank test with $\alpha = 0.05$, we were able to confirm that allotments from NSGA-P statistically significantly outperform those from either Minimum Cores or Maximum Cores in terms of inference time reduction. NSGA-P allotments are also statistically significantly better than Maximum Cores allotments for reducing energy consumption. Although the particular NSGA-P allotments reported here prioritise inference time reduction and therefore consume slightly more energy than Minimum Cores, alternative allotments produced by NSGA-P allow a trade-off providing better energy cost reduction; we refer to Appendix A.2 for further details.

5.4 Impact of Ensemble Heterogeneity on Allotment

We analysed how the structure of the ensemble affects the inference time reduction when using the NSGA-P and Minimum Cores allotment strategies. Specifically, we investigated the relationship

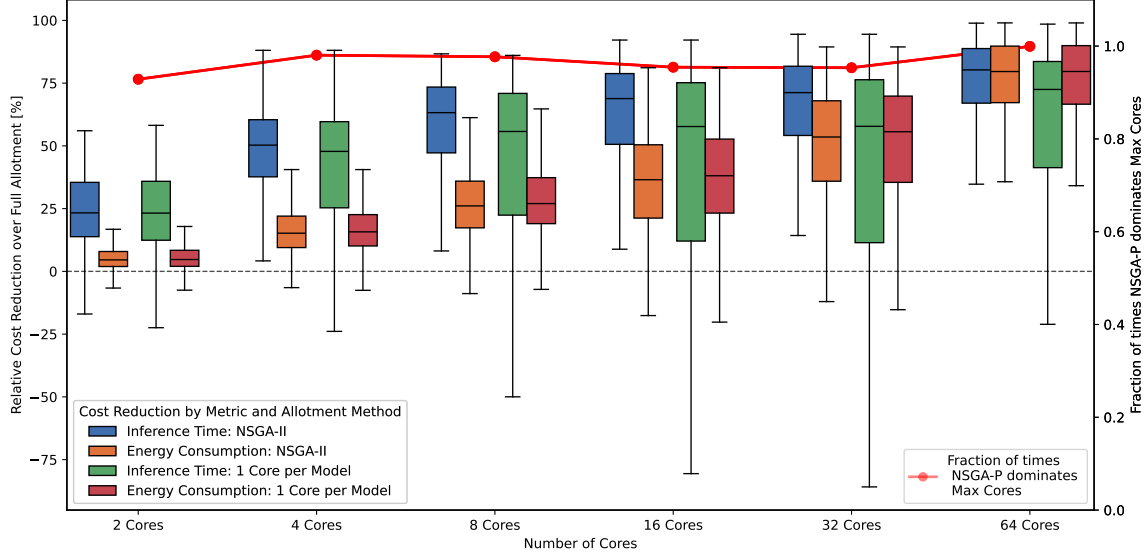


Figure 3: Left y -axis and legend: Comparison of the Minimum Cores and NSGA-P allotment strategies in terms of the average cost reduction, quantified with respect to both inference time and energy consumption, to the Maximum Cores allotment baseline employed by AutoML systems. We report the allotment that exhibits the best inference time reduction and its respective energy cost reduction. Right y -axis and legend: Fraction of ensembles where NSGA-P finds a strictly better allotment than Maximum Cores does. All results were obtained for ensembles of more than one member generated by GES using models generated with SMAC. All results are aggregated across all datasets and random seeds. For visual clarity, outliers are omitted from the plot, not from the computation. See Appendix A for the plot including outliers.

between the heterogeneity among individual ensemble members and the improvement achieved by NSGA-P over Minimum Cores. Figure 4 visualises this relationship. We measured ensemble heterogeneity using the coefficient of variation between the single-core inference times of individual ensemble members and grouped the results into tertiles. We observe a linear relationship between increasing heterogeneity and the gap in inference time reduction between NSGA-P and Minimum Cores allotments across different core counts. We quantified this linear relationship using the Pearson correlation coefficient, as shown in Appendix E. While highly heterogeneous ensembles benefit more from NSGA-P at higher core counts, as the core count increases, ensembles with moderate or low heterogeneity also increasingly benefit from NSGA-P, compared to the Minimum Cores strategy.

6 Conclusions

We investigated the problem of efficient ensemble inference in AutoML by framing it as a moldable task scheduling problem, where both the allotment of cores to individual ensemble members and their execution order are jointly optimised. Our key insight is that naïve maximum-core allocation, which is the implicit default in most AutoML frameworks, is frequently suboptimal, as parallel scaling of machine learning inference is sublinear, and homogeneous allocation can create unnecessary job stalling.

To improve upon this, we proposed NSGA-P, an NSGA-II-based multi-objective allotment strategy that searches over per-member core counts to approximate the best execution schedule for the ensemble, evaluated entirely in simulation using offline resource measurements. This

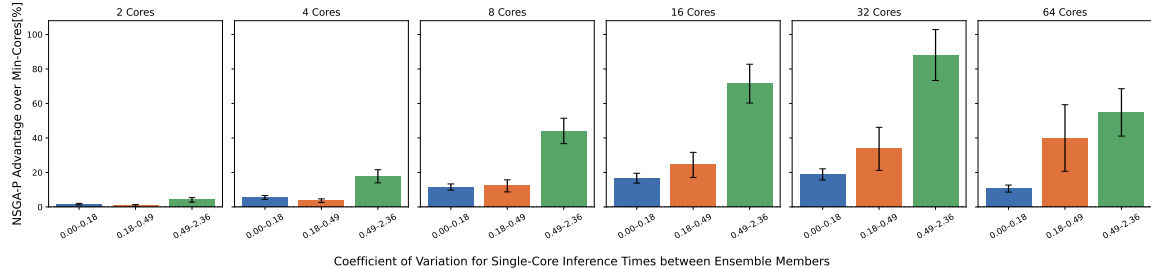


Figure 4: Gap between inference time reduction of NSGA-P allotment over Minimum Cores allotment (y -axis) across different core counts. Gaps are grouped in tertiles by the heterogeneity between ensemble members. Heterogeneity is measured using the coefficient of variation for single-core inference times between ensemble members (higher values indicate more heterogeneous ensembles).

simulation-based approach incurs only a one-time profiling overhead that is negligible relative to training time and becomes even more so for ensembles used repeatedly in production.

Our experiments on 104 classification and regression datasets from the AMLB benchmark suite yield three main findings. Firstly, parallelisation exhibits a clear trade-off: additional cores reduce inference time only up to a point, while energy consumption continues to scale nearly linearly, making core-count selection a non-trivial optimisation problem. Secondly, resource measurements collected during ensembling transfer reliably to prediction workloads ($R^2 > 0.91$), demonstrating the practicality of offline simulation for scheduling. Thirdly, across all core counts we evaluated, NSGA-P outperforms the naïve maximum-core baseline in over 96% of ensembles we considered, and achieves up to 74% lower inference time and energy consumption at 64 cores. While the simpler minimum-cores strategy achieves comparable mean reductions in energy consumption, it yields substantially smaller and more variable reductions of ensemble inference time, particularly for small or heterogeneous ensembles, making NSGA-P a more reliable choice across different deployment scenarios. This is further supported by the fact that NSGA-P often identifies diverse allotments, illustrating its ability to explore structurally different time-energy trade-offs. We note that our evaluation is entirely simulation-based; in future work, our results should be validated under real parallel execution with resource contention and dynamic voltage and frequency scaling (DVFS) effects (*cf* Appendix F).

These results demonstrate that integrating principled, multi-objective parallel resource allocation into AutoML pipelines yields substantial and practical gains, at negligible overhead. Future work should investigate how NSGA-P interacts with other methods similar to post-hoc ensembling, such as multi-layer stacking. We also see a promising direction in extending the approach to heterogeneous hardware configurations and hybrid (GPU+CPU)-based inference with real integration into AutoML systems.

Acknowledgements. Holger Hoos gratefully acknowledges support through an Alexander-von-Humboldt Professorship in Artificial Intelligence. We thank Christian Wassermann for setting up power meters. We would also like to thank the reviewers who gave valuable feedback.

References

- AMD (2024). *AMD64 Architecture Programmer’s Manual Volumes 1–5*. URL: https://docs.amd.com/v/u/en-US/40332_4.09_APM_PUB.
- Benoit, A., V. Le Fèvre, L. Perotin, P. Raghavan, Y. Robert, and H. Sun (2022). “Resilient Scheduling of Moldable Parallel Jobs to Cope With Silent Errors”. In: *IEEE Transactions on Computers* 71.7, pp. 1696–1710.

- Caruana, R., A. Niculescu-Mizil, G. Crew, and A. Ksikes (2004). “Ensemble Selection from Libraries of Models”. In: *Proceedings of the International Conference on Machine Learning*. ICML’04, p. 18.
- Chen, T. and C. Guestrin (2016). “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD’16, pp. 785–794.
- Deb, K., A. Pratap, S. Agarwal, and T. Meyarivan (2002). “A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II”. In: *IEEE Transactions on Evolutionary Computation* 6.2, pp. 182–197.
- Drozdowski, M. (1996). “Scheduling Multiprocessor Tasks – An Overview”. In: *European Journal of Operational Research* 94.2, pp. 215–230.
- Du, J. and J. Y.-T. Leung (1989). “Complexity of Scheduling Parallel Task Systems”. In: *SIAM Journal on Discrete Mathematics* 2.4, pp. 473–487.
- Dutot, P.-F., G. Mounié, and D. Trystram (2004). “Scheduling Parallel Tasks: Approximation Algorithms”. In: *Handbook of Scheduling: Algorithms, Models, and Performance Analysis*. Chap. 26, pp. 1–24.
- Erickson, N., J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li, and A. J. Smola (2020). “AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data”. In: *Proceedings of the International Conference on Machine Learning*. ICML’20.
- Erickson, N., L. Purucker, A. Tschalzev, D. Holzmüller, P. M. Desai, D. Salinas, and F. Hutter (2025). “TabArena: A Living Benchmark for Machine Learning on Tabular Data”. In: *Proceedings of the 39th Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. NeurIPS’25.
- Feurer, M., K. Eggensperger, S. Falkner, M. Lindauer, and F. Hutter (2022). “Auto-sklearn 2.0: Hands-Free AutoML via Meta-Learning”. In: *Journal of Machine Learning Research* 23.261, pp. 11936–11996.
- Garey, M. R. and R. L. Graham (1975). “Bounds for Multiprocessor Scheduling with Resource Constraints”. In: *SIAM Journal on Computing* 4.2, pp. 187–200.
- Gijsbers, P., M. L. P. Bueno, S. Coors, E. LeDell, S. Poirier, J. Thomas, B. Bischl, and J. Vanschoren (2024). “AMLB: An AutoML Benchmark”. In: *Journal of Machine Learning Research* 25.101, pp. 1–65.
- Gorishniy, Y., A. Kotelnikov, and A. Babenko (2025). “TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling”. In: *Proceedings of the International Conference on Learning Representations*. ICLR’25.
- Hernandez, J. G., A. K. Saini, A. Ghosh, and J. H. Moore (2025). “The Tree-Based Pipeline Optimization Tool: Tackling Biomedical Research Problems with Genetic Programming and Automated Machine Learning”. In: *Patterns* 6.7, p. 101314.
- Holzmüller, D., L. Grinsztajn, and I. Steinwart (2024). “Better by Default: Strong Pre-Tuned MLPs and Boosted Trees on Tabular Data”. In: *Proceedings of the International Conference on Neural Information Processing Systems*. NeurIPS’24, pp. 26577–26658.
- Hoos, H. H., R. Kaminski, T. Schaub, and M. Schneider (2012). “aspeed: ASP-based Solver Scheduling”. In: *Technical Communications of the 28th International Conference on Logic Programming*. ICLP’12, pp. 176–187.
- Jansen, K. (2012). “A(3/2+ ϵ) Approximation Algorithm for Scheduling Moldable and Non-Moldable Parallel Tasks”. In: *Proceedings of the Annual ACM Symposium on Parallelism in Algorithms and Architectures*. SPAA’12, pp. 224–235.
- Kocher, N., C. Wassermann, L. Hennig, J. Seng, H. Hoos, K. Kersting, M. Lindauer, and M. Müller (2025). “Guidelines for the Quality Assessment of Energy-Aware NAS Benchmarks”. In: *Proceedings of the International Symposium on Cluster, Cloud and Internet Computing*. CCGRIDW’25, pp. 50–59.
- LeDell, E. and S. Poirier (2020). “H2O AutoML: Scalable Automatic Machine Learning”. In: *Proceedings of the International Conference on Machine Learning*. ICML’20.

- Lindauer, M., K. Eggensperger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, and F. Hutter (2022). “SMAC3: A Versatile Bayesian Optimization Package for Hyperparameter Optimization”. In: *Journal of Machine Learning Research* 23.54, pp. 1–9.
- Maier, J., F. Möller, and L. O. Purucker (2024). “Hardware-Aware Ensemble Selection for Balancing Predictive Accuracy and Cost”. In: *Proceedings of the International Conference on Automated Machine Learning*. AutoML’24.
- Purucker, L. O., L. Schneider, M. Anastacio, J. Beel, B. Bischl, and H. Hoos (2023). “Q(D)O-ES: Population-based Quality (Diversity) Optimisation for Post-Hoc Ensemble Selection in AutoML”. In: *Proceedings of the International Conference on Automated Machine Learning*. AutoML’23, pp. 1–34.
- Schöne, R., T. Ilsche, M. Bielert, M. Velten, M. Schmidl, and D. Hackenberg (2021). “Energy Efficiency Aspects of the AMD Zen 2 Architecture”. In: *Proceedings of the International Conference on Cluster Computing*. CLUSTER’21, pp. 562–571.
- Thornton, C., F. Hutter, H. H. Hoos, and K. Leyton-Brown (2013). “Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms”. In: *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD’13, pp. 847–855.
- Turek, J., J. L. Wolf, and P. S. Yu (1992). “Approximate Algorithms Scheduling Parallelizable Tasks”. In: *Proceedings of the Annual ACM Symposium on Parallel Algorithms and Architectures*. SPAA’92, pp. 323–332.
- Villarrubia, J., L. Costero, F. D. Igual, and K. Olcoz (2025). “Leveraging Multi-Instance GPUs through moldable task scheduling”. In: *Journal of Parallel and Distributed Computing* 204, p. 105128.

Submission Checklist

1. For all authors...

- (a) Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope? [Yes] We state in the abstract and introduction that we compare NSGA-P to the Maximum Cores and Minimum Cores allotment strategies, which we subsequently do, and that smart allocation and scheduling of ensemble members can be crucial to improve efficiency of AutoML system, which is supported by our results showing great performance of NSGA-P.
- (b) Did you describe the limitations of your work? [Yes] See Appendix F.
- (c) Did you discuss any potential negative societal impacts of your work? [Yes] See Appendix G.
- (d) Did you read the ethics review guidelines and ensure that your paper conforms to them? (see <https://2022.automl.cc/ethics-accessibility/>) [Yes] To the best of our knowledge, our paper conforms to them.

2. If you ran experiments...

- (a) Did you use the same evaluation protocol for all methods being compared (e.g., same benchmarks, data (sub)sets, available resources, etc.)? [Yes] All compared allotment methods were run on the same number of datasets and had the same amount of resources. See Section 4 and Appendix B.
- (b) Did you specify all the necessary details of your evaluation (e.g., data splits, pre-processing, search spaces, hyperparameter tuning details and results, etc.)? [Yes] We used the same data splits, preprocessing methods and search spaces across all three allotment methods. Hyperparameters were not tuned for the allotment methods. See Section 4 and Appendix B.
- (c) Did you repeat your experiments (e.g., across multiple random seeds or splits) to account for the impact of randomness in your methods or data? [Yes] Ensemble member generation and ensembling, as well as the subsequent scheduling and allotment, were repeated across 5 seeds.
- (d) Did you report the uncertainty of your results (e.g., the standard error across random seeds or splits)? [Yes] The uncertainty is included in the variance given in the per-core allotment results, which we reported; see Section 5.
- (e) Did you report the statistical significance of your results? [Yes] We reported the Wilcoxon signed-rank test with $\alpha = 0.05$ in Section 5.
- (f) Did you use enough repetitions, datasets, and/or benchmarks to support your claims? [Yes] We empirically evaluated our approach using the standard AMLB benchmark set of 104 datasets using 5 random seeds achieving statistical significance. We believe this to support our claims.
- (g) Did you compare performance over time and describe how you selected the maximum runtime? [No] We did not compare performance over time. Maximum running time for SMAC was chosen to be eight hours to allow for sufficiently well-performing models to be generated.
- (h) Did you include the total amount of compute and the type of resources used (e.g., type of GPUs, internal cluster, or cloud provider)? [Yes] We reported the hardware platform we ran our experiments on and the time duration of the most time intensive tasks, *i.e.*, the ensemble member generation stage. We did not explicitly calculate the total compute amount used.

- (i) Did you run ablation studies to assess the impact of different components of your approach? [\[Yes\]](#) In Appendix A.1, we provide ablations for the heuristic used for scheduling allotted ensembles.
3. With respect to the code used to obtain your results...
- (a) Did you include the code, data, and instructions needed to reproduce the main experimental results, including all dependencies (e.g., `requirements.txt` with explicit versions), random seeds, an instructive README with installation instructions, and execution commands (either in the supplemental material or as a URL)? [\[Yes\]](#) See our repository linked in Section 1.
 - (b) Did you include a minimal example to replicate results on a small subset of the experiments or on toy data? [\[Yes\]](#) See instructions in the linked repository.
 - (c) Did you ensure sufficient code quality and documentation so that someone else can execute and understand your code? [\[Yes\]](#) We believe that our code base has sufficient quality and documentation.
 - (d) Did you include the raw results of running your experiments with the given code, data, and instructions? [\[Yes\]](#) See our repository linked in Section 1.
 - (e) Did you include the code, additional data, and instructions needed to generate the figures and tables in your paper based on the raw results? [\[Yes\]](#) See our repository linked in Section 1.
4. If you used existing assets (e.g., code, data, models)...
- (a) Did you cite the creators of used assets? [\[Yes\]](#) See Section 4.
 - (b) Did you discuss whether and how consent was obtained from people whose data you’re using/curating if the license requires it? [\[N/A\]](#) We only used publicly available datasets from OpenML.
 - (c) Did you discuss whether the data you are using/curating contains personally identifiable information or offensive content? [\[N/A\]](#) We only used publicly available datasets. To the best of our knowledge, these do not identify or offend anybody.
5. If you created/released new assets (e.g., code, data, models)...
- (a) Did you mention the license of the new assets (e.g., as part of your code submission)? [\[Yes\]](#) See our repository linked in Section 1.
 - (b) Did you include the new assets either in the supplemental material or as a URL (to, e.g., GitHub or Hugging Face)? [\[Yes\]](#) See our repository linked in Section 1.
6. If you used crowdsourcing or conducted research with human subjects...
- (a) Did you include the full text of instructions given to participants and screenshots, if applicable? [\[N/A\]](#) No participants.
 - (b) Did you describe any potential participant risks, with links to institutional review board (IRB) approvals, if applicable? [\[N/A\]](#) No participants.
 - (c) Did you include the estimated hourly wage paid to participants and the total amount spent on participant compensation? [\[N/A\]](#) No participants.
7. If you included theoretical results...

- (a) Did you state the full set of assumptions of all theoretical results? [N/A] We do not provide theoretical results in this paper.
- (b) Did you include complete proofs of all theoretical results? [N/A] See above.

A Additional Results

This section outlines additional results of the main experiments. First, we show results accompanying those presented in Figure 3. Table 1 presents percentages of mean reductions over the Maximum Core baselines for two allotment strategies (Minimum Cores and NSGA-P) and for both objectives (time and energy), with associated standard deviations, across different core counts. Figure 5 visualises the same results as Figure 3, this time with outliers included.

The outliers in Figure 5 show the high variability of the Minimum Cores strategy, particularly for inference time reduction at larger core counts. These outliers mainly occur for small or heterogeneous ensembles containing long-running bottleneck models. Restricting all models to a single core can cause such models to dominate the makespan, leaving many cores idle and substantially reducing parallel efficiency. This behaviour is reflected in the large standard deviations of Minimum Cores reported in Table 1, which increase considerably with higher core counts. In contrast, NSGA-P exhibits substantially lower variance and more stable performance across different ensemble structures. By selectively assigning additional cores to slow members to bring their running time in line with the rest of the ensemble, NSGA-P is able to avoid the catastrophic failure mode. This means that NSGA-P consistently identifies more balanced, robust and reliable allotments.

Table 1: Mean inference time and energy consumption reductions of NSGA-P and Minimum Cores over Maximum Cores. Results were obtained for ensembles generated by GES using models generated via SMAC and are aggregated across all datasets and random seeds. NSGA-P allotments minimise inference time. Standard deviations are noted in brackets.

Number of Cores	Mean Reduction [%] (Min Cores)		Mean Reduction [%] (NSGA-P)	
	Inference Time	Energy Consumption	Inference Time	Energy Consumption
2	21.84 (18.21)	5.28 (6.12)	23.49 (15.65)	5.24 (5.92)
4	38.75 (32.68)	16.39 (10.82)	46.08 (19.76)	16.12 (10.38)
8	38.42 (50.42)	27.89 (15.33)	56.33 (22.82)	26.95 (14.94)
16	31.11 (73.17)	37.26 (21.14)	60.58 (26.32)	35.97 (20.64)
32	26.96 (92.50)	50.62 (24.41)	62.69 (27.00)	50.24 (23.44)
64	46.53 (95.43)	73.82 (22.95)	73.73 (22.16)	74.45 (21.65)

A.1 Choice of the Scheduling Heuristic

In addition to the allotment strategy, the quality of the final ensemble execution depends on the scheduling heuristic used to order already allotted ensemble members. Since the underlying scheduling problem for moldable tasks is $\mathcal{NP}$ -hard, practical systems typically rely on heuristic scheduling rules. We therefore ablated the choice of list scheduling heuristic used to order allotted ensemble members for execution. We compared three priority criteria: LPT, which schedules tasks in decreasing order of their (single-core) inference time; Highest Workload First (HWL), which orders tasks by the product of inference time and allocated core count; and Highest Core Count First (HCC), which prioritises tasks with larger core allocations. All three are standard list scheduling criteria (Dutot et al., 2004).

Figure 6 compares the resulting schedules across total core counts of 2, 4, and 8 cores, in terms of average idle time per core, where lower values indicate more efficient utilisation of available resources. Across all evaluated core counts, LPT consistently achieves the median lowest idle time, particularly at larger core counts. At 2 cores, HWL and HCC perform comparably to LPT, with HCC exhibiting a slightly worse median idle time and larger variance. This suggests that scheduling order matters little when parallelism is limited. As the core count increases, idle times produced by all three heuristics become more variable, with HWL also becoming less efficient.

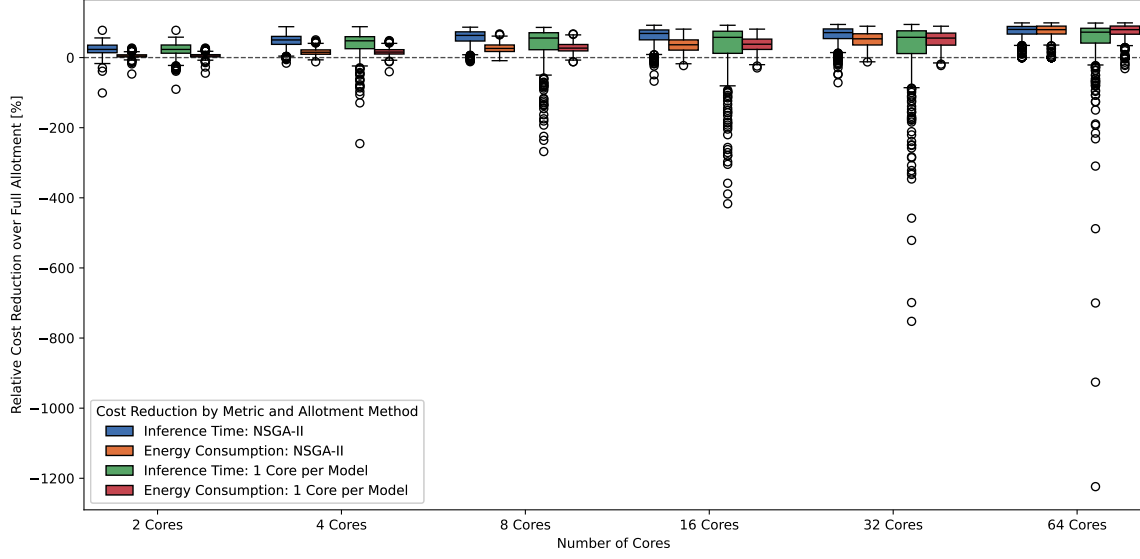


Figure 5: Left y -axis and legend: Comparison of the Minimum Cores and NSGA-P allotment strategies in terms of the average cost reduction, quantified with respect to both inference time and energy consumption, to the Maximum Cores allotment baseline employed by AutoML systems. We report the allotment that exhibits the best inference time reduction and its respective energy cost reduction. Right y -axis and legend: Fraction of ensembles where NSGA-P finds a strictly better allotment than Maximum Cores does. All results were obtained for ensembles of more than one member generated by GES using models generated with SMAC. All results are aggregated across all datasets and random seeds. Outliers are **included**.

HCC consistently produces the worst-performing schedules, leading to substantially higher idle times due to poor balancing of short and long-running tasks.

These results are consistent with empirical evidence that placing long tasks first minimises the risk of a single long task stalling execution. Since minimising makespan for a fixed allotment also reduces idle-time energy consumption, we adopted LPT as the default scheduling heuristic throughout the main experiments.

A.2 Alternative Trade-off: Prioritising Energy Consumption

The main experiments in Section 5 report the NSGA-P allotments that achieve the largest reductions in inference time. However, since NSGA-II approximates diverse latency–energy trade-offs, alternative allotments can instead prioritise reductions in energy consumption. We therefore present results for the NSGA-P allotment achieving the largest reduction in energy consumption in Figure 7 (equivalent to Figure 3 in the main paper), Figure 8 (with outliers included) and Table 2 (percentages of mean reductions). Compared with the inference time-oriented allotments discussed in the main paper, energy-oriented allotments generally allocate fewer cores to individual ensemble members and thereby reduce overall parallelism. Consequently, they achieve slightly stronger reductions in energy consumption while sacrificing some inference-time improvements.

Even under this alternative objective, NSGA-P continues to outperform the Maximum Cores baseline consistently across nearly all evaluated ensembles and core counts. At 64 available cores, NSGA-P achieves average reductions of 74.89% in energy consumption while still reducing inference time by 70.74%. In comparison, Minimum Cores achieves similar average energy savings but exhibits substantially larger variance in inference time reductions, particularly for heterogeneous ensembles containing long-running models. These results demonstrate that NSGA-P exposes a flexible trade-off that can be adapted to different practical requirements. Depending on whether latency or energy

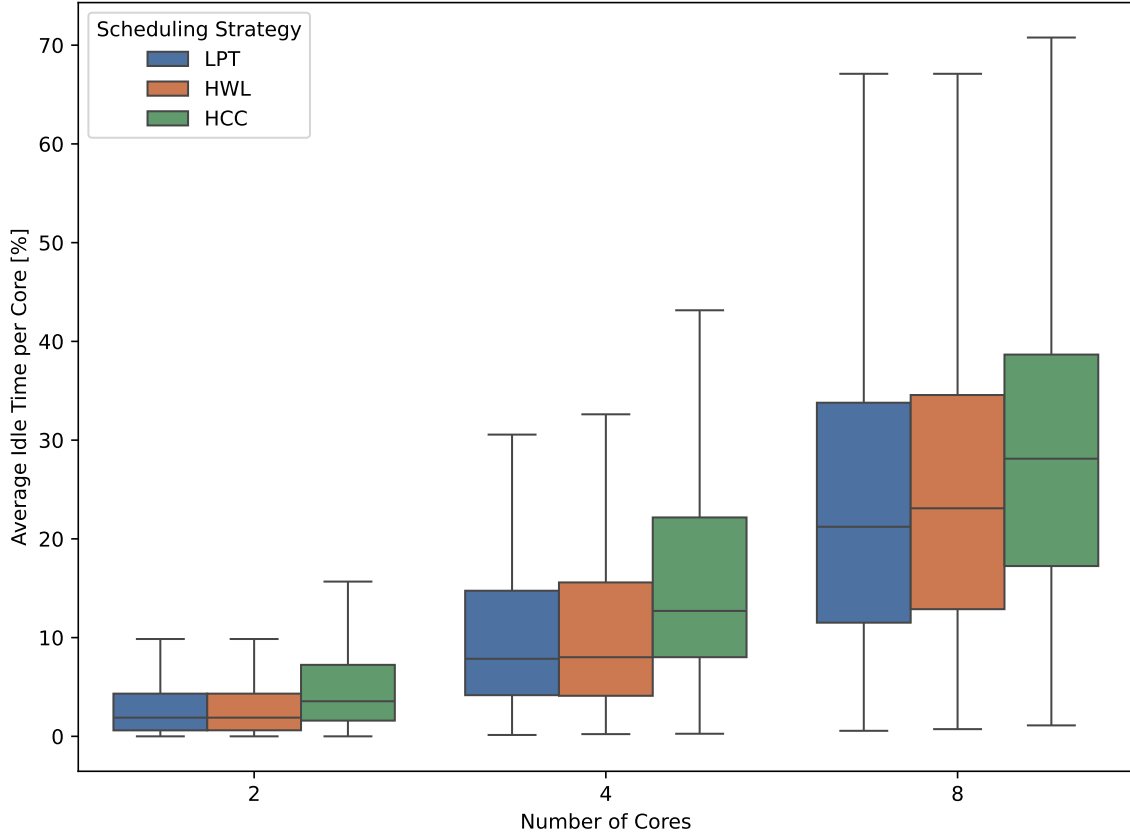


Figure 6: Comparison of different heuristics for the scheduling of allotted ensembles. We compare Longest Processing Time (LPT), Highest Workload (HWL) and Highest Core Count (HCC). We report average idle time per core. Lower is better.

efficiency is prioritised, different allotments can be selected without modifying the underlying scheduling framework.

Table 2: Mean inference resource reductions of NSGA-P and Minimum Cores over Maximum Cores. Results were obtained for ensembles generated by GES using models generated via SMAC and are aggregated across all datasets and random seeds. NSGA-P allotments minimise **energy consumption**. Standard deviations are noted in brackets.

Number of Cores	Mean Reduction [%] (Min Cores)		Mean Reduction [%] (NSGA-P)	
	Inference Time	Energy Consumption	Inference Time	Energy Consumption
2	21.84 (18.21)	5.28 (6.12)	22.11 (16.94)	5.43 (5.79)
4	38.75 (32.68)	16.39 (10.82)	42.52 (24.05)	16.69 (10.26)
8	38.42 (50.42)	27.89 (15.33)	48.89 (30.21)	28.21 (14.90)
16	31.11 (73.17)	37.26 (21.14)	50.36 (38.77)	37.88 (20.36)
32	26.96 (92.50)	50.62 (24.41)	54.76 (33.98)	51.66 (23.23)
64	46.53 (95.43)	73.82 (22.95)	70.74 (23.94)	74.89 (21.60)

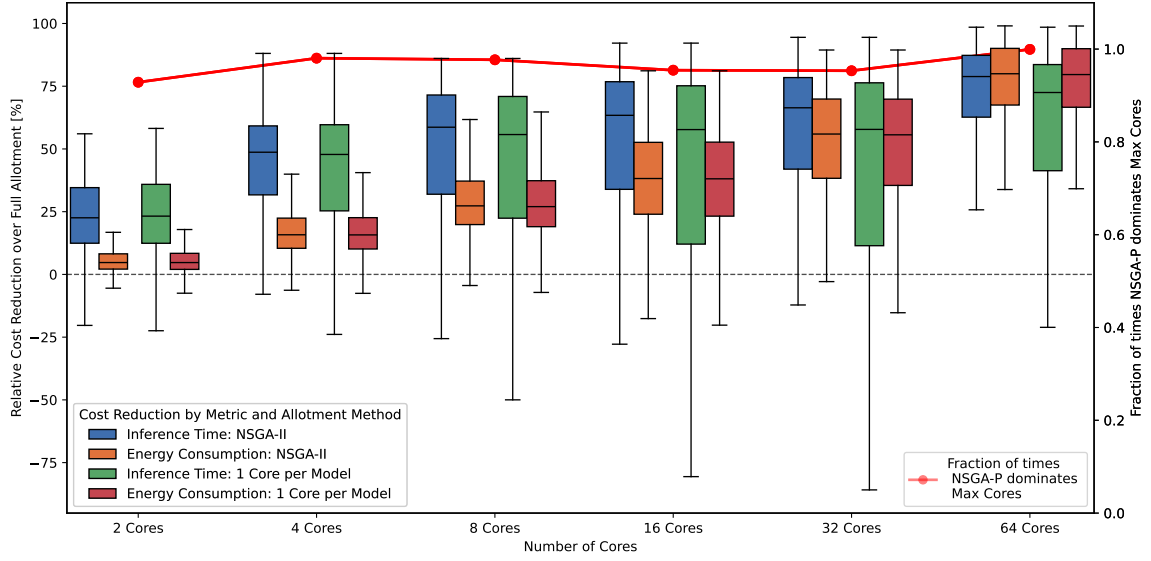


Figure 7: Left y -axis and legend: Comparison of the Minimum Cores and NSGA-P allotment strategies in terms of the average cost reduction, quantified with respect to both inference time and energy consumption, to the Maximum Cores allotment baseline employed by AutoML systems. We report the allotment that exhibits the best **energy consumption** reduction and its respective inference time reduction. Right y -axis and legend: Fraction of ensembles where NSGA-P finds a strictly better allotment than Maximum Cores does. All results were obtained for ensembles of more than one member generated by GES using models generated with SMAC. All results are aggregated across all datasets and random seeds. For visual clarity, outliers are omitted from the plot, not from the computation. See Figure 8 for the plot including outliers.

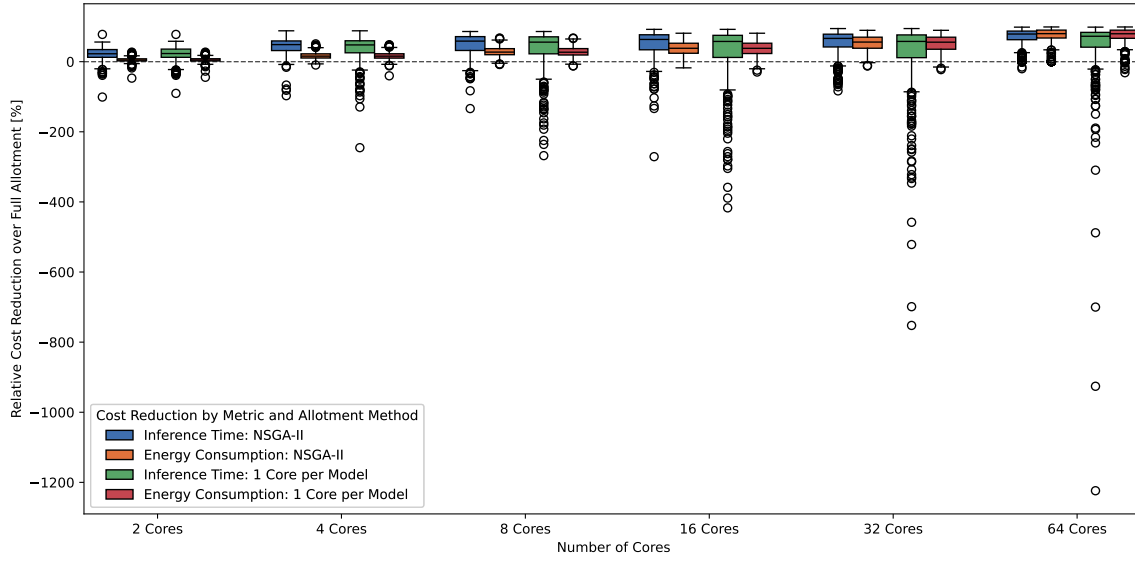


Figure 8: Left y -axis and legend: Comparison of the Minimum Cores and NSGA-P allotment strategies in terms of the average cost reduction, quantified with respect to both inference time and energy consumption, to the Maximum Cores allotment baseline employed by AutoML systems. We report the allotment that exhibits the best **energy consumption** reduction and its respective inference time reduction. Right y -axis and legend: Fraction of ensembles where NSGA-P finds a strictly better allotment than Maximum Cores does. All results were obtained for ensembles of more than one member generated by GES using models generated with SMAC. All results are aggregated across all datasets and random seeds. Outliers are included.

A.3 Resource Consistency for Different Core Counts

Figure 9 presents the consistency of inference time and energy consumption between the ensembling stage and the final prediction, extended to core counts of 2, 4 and 8 from the single-core analysis in Section 5. Resource measurements remain highly correlated between the two stages across all evaluated core counts. We confirm that the offline resource measurement approach remains robust and reliable as more cores are allocated, and that the LPT scheduling heuristic, which relies on training-data inference times as proxies for test-time scheduling, remains well-calibrated across all settings evaluated in this work.

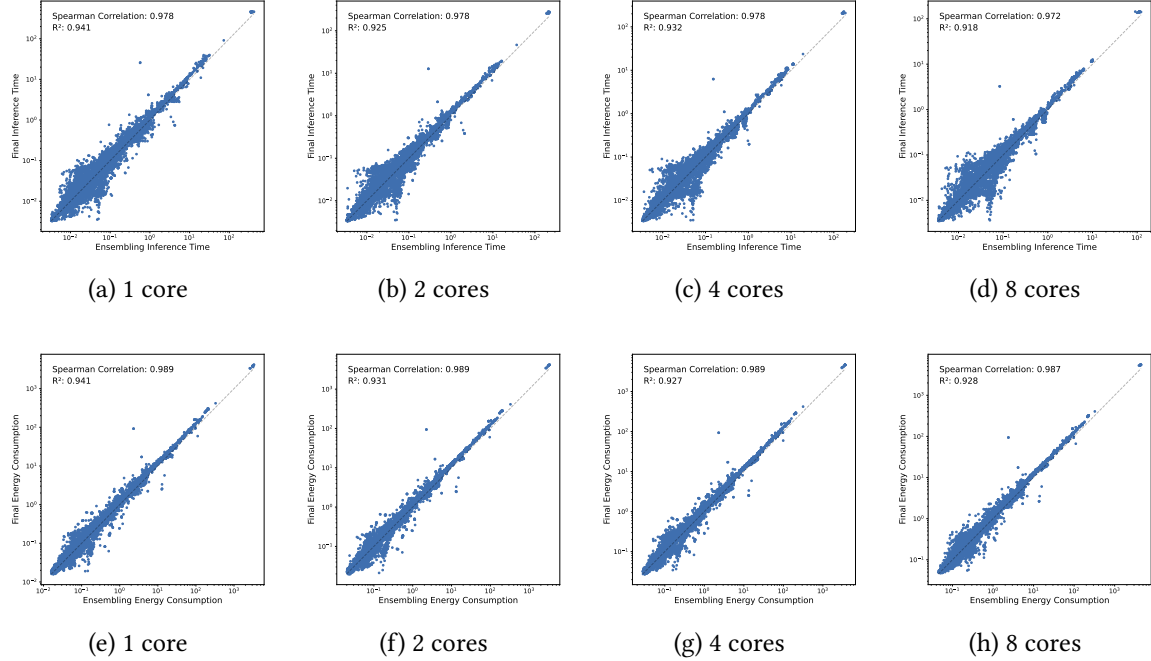


Figure 9: Consistency of inference time and energy consumption between the ensembling stage (on training data) and final predictions (on test data) using different core counts on the AMD EPYC 7543 platform. Values are normalised per model and dataset. Top row: Inference time. Bottom row: Energy consumption.

B Detailed Setup of Experiments

Preprocessing and search spaces of ensemble members. The only preprocessing step incorporated as part of the search space was the balancing method. Here, we considered with equal probability the following options: no balancing, oversampling, undersampling and hybrid sampling. Further preprocessing steps, such as categorical encoding, imputation of missing categorical values or scaling of numerical features, were handled by the respective models. The hyperparameter spaces follow the pytabkit defaults, with minor adaptations for this study, and can be found in Appendix C. As all models employed in the search space require numerical imputation, we incorporate mean imputation for all numerical features by default. The remaining preprocessing steps are handled by the models within the search space.

Resource allocations per dataset. For datasets with large memory footprints or long training times, we allocated additional cores and RAM during the ensemble member generation using SMAC according to a simple heuristic based on dataset size during the ensemble member generation phase. Although the dataset size does not fully capture all factors influencing training time, it effectively

prevents too many out-of-memory errors and thereby improves the robustness of the ensemble member generation phase. Below, we provide an overview of the allocated resources per dataset.

Table 3: Resource allocation per dataset used in the experiments.

Task ID	Optimized Memory Usage [MB]	Assigned Memory [GB]	Assigned CPU cores
167210	0.083	16	1
233211	2.539	16	1
233212	36.744	16	1
233213	191.306	64	4
233214	89.109	32	2
233215	6.347	16	1
317614	168.000	64	4
359929	490.127	64	4
359930	0.070	16	1
359931	0.025	16	1
359932	0.041	16	1
359933	0.137	16	1
359934	0.124	16	1
359935	0.416	16	1
359936	1.527	16	1
359937	7.344	16	1
359938	0.607	16	1
359939	9.631	16	1
359940	9.098	16	1
359941	10.149	16	1
359942	3.030	16	1
359943	39.079	16	1
359944	0.205	16	1
359945	1.045	16	1
359946	3.180	16	1
359948	2.139	16	1
359949	2.196	16	1
359950	0.035	16	1
359951	0.351	16	1
359952	1.914	16	1
360932	23.634	16	1
360933	23.733	16	1
360945	0.647	16	1
2073	0.075	16	1
3945	55.456	16	1
7593	58.703	16	1
10090	60.036	32	2
146818	0.041	16	1
146820	0.179	16	1
167120	9.729	16	1
168350	0.200	16	1
168757	0.070	16	1
168784	0.244	16	1

168868	52.972	16	1
168909	80.171	32	2
168910	26.500	16	1
168911	0.613	16	1
189354	18.987	16	1
189355	107.435	32	2
189356	311.502	64	4
189922	12.288	16	1
190137	0.773	16	1
190146	0.076	16	1
190392	3.307	16	1
190410	7.285	16	1
190411	0.867	16	1
190412	4.002	16	1
211979	19.511	16	1
211986	11.436	16	1
359953	2.982	16	1
359954	0.074	16	1
359955	0.025	16	1
359956	0.191	16	1
359957	3.718	16	1
359958	0.241	16	1
359959	0.052	16	1
359960	0.045	16	1
359961	1.764	16	1
359962	0.213	16	1
359963	0.189	16	1
359964	0.709	16	1
359965	0.186	16	1
359966	5.891	16	1
359967	26.711	16	1
359968	0.429	16	1
359969	1.353	16	1
359970	1.433	16	1
359971	0.534	16	1
359972	0.497	16	1
359973	34.960	16	1
359974	0.300	16	1
359975	0.822	16	1
359976	220.712	64	4
359977	4.010	16	1
359979	4.070	16	1
359980	13.872	16	1
359981	1.838	16	1
359982	2.450	16	1
359983	2.413	16	1
359984	8.170	16	1
359985	42.976	16	1
359986	288.172	64	4
359987	3.075	16	1

359988	344.020	64	4
359989	344.020	64	4
359990	28.224	16	1
359991	7.437	16	1
359992	25.067	16	1
359993	4.605	16	1
359994	82.590	32	2
360112	754.381	64	4
360113	90.514	32	2
360114	129.000	64	4
360975	2965.362	64	4

C Search Spaces

The search spaces are primarily based on the search spaces provided by pytabkit (Holzmüller et al., 2024). Below, we give an overview of the search space used in this work.

Table 4: CASH Search Space

Hyperparameter	Type	Range / Choices	Default
Top-level hyperparameters			
model	Categorical	{realmlp, tabm, xgboost}	xgboost
balancing (active only if classification)	Categorical	{upsampling, downsampling, hybrid, none}	downsampling
RealMLP hyperparameters			
num_emb_type	Categorical	{none, pbld, pl, plr}	none
add_front_scale	Categorical	{True, False}	True
n_hidden	Int (log)	[64, 512]	256
n_layers	Int	[1, 5]	3
lr	Float (log)	[1e-2, 5e-1]	1e-1
p_drop	Float	[0, 0.6]	0.3
wd	Categorical	{low, high}	low
wd_low (active only if wd = low)	Float	[0, 1e-3]	0
wd_high (active only if wd = high)	Float (log)	[1e-3, 1e-1]	1e-3
plr_sigma	Float (log)	[1e-2, 1e2]	1e-1
act	Categorical	{relu, selu, mish, silu, gelu}	relu
use_parametric_act	Categorical	{False, True}	False
p_drop_sched	Categorical	{flat_cos, constant}	flat_cos
wd_sched	Categorical	{flat_cos, constant}	flat_cos
ls_eps	Categorical	{zero, positive}	zero
ls_eps_pos (active only if ls_eps = positive)	Float	[0, 0.2]	0.0
lr_sched	Categorical	{coslog4, cos}	cos
sq_mom	Float (log)	[0.9, 0.999]	0.9

Hyperparameter	Type	Range / Choices	Default
plr_lr_factor	Float	[0.03, 0.3]	0.03
XGBoost hyperparameters			
n_estimators	Int (log)	[2, 2000]	1000
max_depth	Int	[2, 10]	3
subsample	Float	[1e-3, 1]	1
colsample_bytree	Float	[0.1, 1]	1
colsample_bylevel	Float	[0.1, 1]	1
min_child_weight	Float (log)	[e ⁻¹⁶ , e ⁵]	1
reg_alpha	Categorical	{zero, positive}	zero
reg_alpha_pos (active only if reg_alpha = positive)	Float (log)	[e ⁻¹⁶ , e ²]	1
reg_lambda	Categorical	{zero, positive}	zero
reg_lambda_pos (active only if reg_lambda = positive)	Float (log)	[e ⁻¹⁶ , e ²]	1
reg_gamma	Categorical	{zero, positive}	zero
reg_gamma_pos (active only if reg_gamma = positive)	Float (log)	[e ⁻¹⁶ , e ²]	1
TabM hyperparameters			
lr	Float (log)	[1e-4, 3e-3]	1e-3
wd	Categorical	{zero, positive}	zero
wd_pos (active only if wd = positive)	Float (log)	[1e-4, 1e-1]	1e-3
n_blocks	Int	[1, 5]	3
d_block	Int	[32, 512]	128
dropout	Float	[0, 0.5]	0
d_embedding	Int	[8, 32]	16
num_emb_n_bins	Int	[2, 128]	32

D Running Time

In Figure 10 we highlight the running time required by NSGA-P to perform allotment for one ensemble. We note that this does not include profiling costs for the individual models at different core counts, as we collect these beforehand.

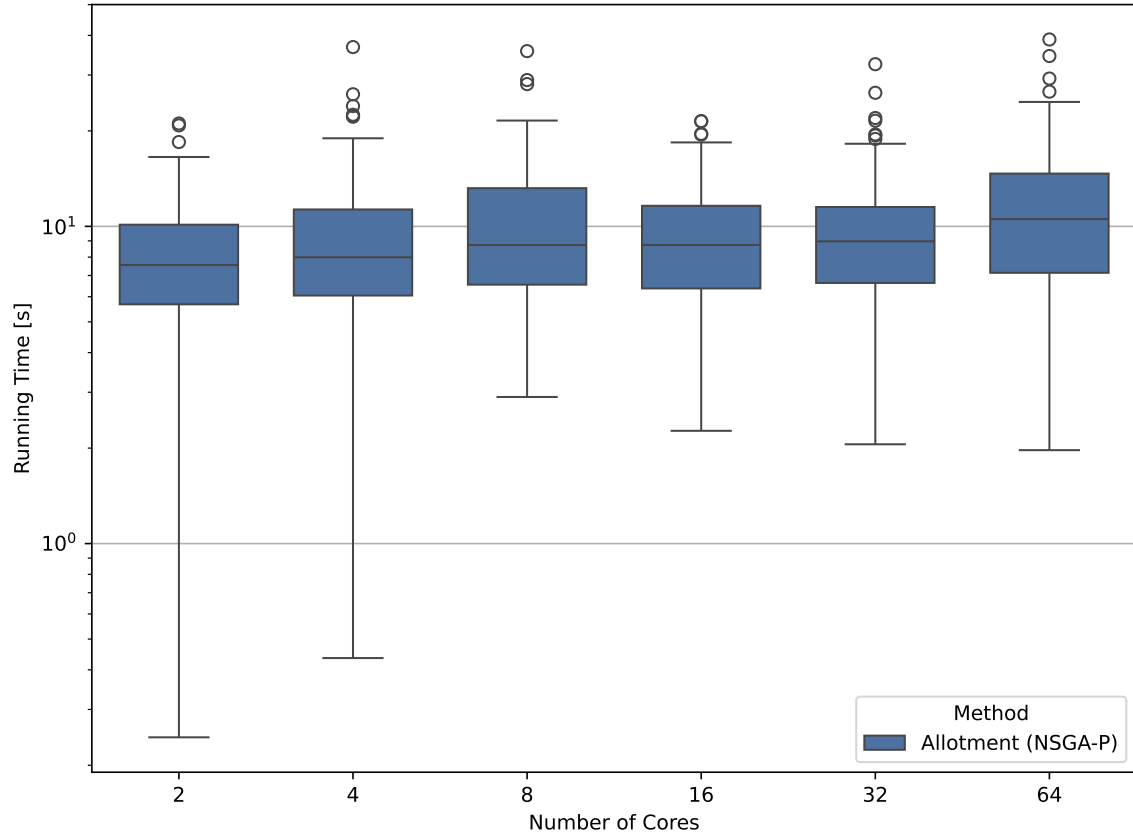


Figure 10: Running time required by NSGA-P to perform allotment on one ensemble.

E Ensemble Heterogeneity

We report the values of the Pearson correlation coefficient between the coefficient of variance for the single-core running time of individual ensemble members and the improvement gap between the NSGA-P allotment and Minimum Cores allotment in Table 5.

Cores	2	4	8	16	32	64
Pearson r	0.33	0.44	0.48	0.46	0.38	0.14

Table 5: Pearson correlation coefficient for the heterogeneity experiments performed in Section 5.4.

F Limitations

Our work has the following limitations: (i) Ensembles are only generated from one search space, specified in Appendix C, due to budget constraints. Results might vary for very different model combinations, even though we included different model families. (ii) We analysed performance analytically offline. Future work needs to implement a real process-based environment. (iii) We do not evaluate hybrid GPU+CPU inference involving state-of-the-art foundation models. (iv) Offline simulation assumes that measured inference times and energy values are stationary at deployment. It does not model dynamic voltage and frequency scaling, memory-bandwidth contention between concurrently executing members, or I/O bottlenecks, all of which can affect real parallel execution. (v) Energy measurements are obtained from per-core CPU counters and exclude memory and I/O consumption. The reported reductions therefore represent a lower bound on total system energy and may shift for memory-bound workloads. We note that it is not trivial to get accurate energy measurements, including I/O consumption, for workload on core level, precisely because of the resource contention. We also considered conducting experiments using power meters, which are accurate enough for deep learning pipelines (Kocher et al., 2025). However, a major bottleneck is their sampling rate, as it is too low to accurately capture the power consumption of many classical machine learning pipelines.

G Broader Impact Statement

We believe that our work is methodological and analytical in nature. Thus, after careful reflection, we determined that this work does not present significant or new negative impacts to society or the environment that are not already present for existing state-of-the-art AutoML systems. We proposed to efficiently schedule ensembles created by AutoML systems while keeping predictive performance consistent and reducing resource consumption. Therefore, we only see the potential for a positive impact in our contribution.